



Projet SEPA26

Service REST de Gestion de Transactions SEPA

Implémentation partielle de la norme ISO 20022

Informations du projet

Auteur : Ahcène AMOUCHAS

Formation : Master 1 GIL

Module : Langage Web 2 — XML

Encadrant : S. Hérauville

Version : v1.0

Année : 2025/2026

Université de Rouen Normandie

UFR Sciences et Techniques

Date de rendu : 22 mai 2026

Table des matières

1	Introduction et présentation du projet	3
1.1	Contexte	3
1.2	Objectifs	3
1.3	Architecture générale	3
1.4	Technologies utilisées	4
2	Modélisation XML et validation XSD	5
2.1	Structure d'un flux SEPA26	5
2.2	Le schéma XSD	5
2.2.1	Rôle et conception	5
2.2.2	Types simples définis	5
2.2.3	Namespace et espace de noms	6
2.2.4	Validation dans le service	6
3	Modèle de données Java et persistance JPA	7
3.1	Mapping objet-relationnel	7
3.1.1	Les entités principales	7
3.1.2	Relations entre entités	7
3.1.3	Double annotation JPA + JAXB	7
3.1.4	Ordre des éléments XML avec @XmlType	8
4	Service REST — Endpoints et contrôleurs	9
4.1	Vue d'ensemble des endpoints	9
4.2	Contrôleurs	9
4.2.1	GetController — Consultation des données	9
4.2.2	PostController — Insertion avec validation	11
4.2.3	DeleteController — Suppression avec nettoyage	11
5	Transformation XSLT	13
5.1	Rôle de la feuille de style	13
5.2	Fonctionnalités clés	13
5.2.1	Gestion du namespace	13
5.2.2	Date dynamique avec XSLT 3.0	13
5.2.3	Gestion du choix BIC / Agent	14
5.2.4	Tri et sommaire	14

6	Déploiement et intégration continue	15
6.1	Conteneurisation Docker	15
6.2	Docker Compose	15
6.3	Pipeline Jenkins	16
7	Fonctionnalités bonus	17
7.1	API de recherche	17
7.2	Gestion enrichie des erreurs	17
7.3	Outil de transfert de fichiers	18
7.4	Journalisation (Logging)	18
8	Tests et validation Postman	19
8.1	Tests unitaires	19
8.2	Validation Postman	19
9	Conclusion	21
9.1	Bilan des réalisations	21
9.2	Difficultés rencontrées	21
9.3	Perspectives d'amélioration	22

1. Introduction et présentation du projet

1.1 Contexte

Ce projet s'inscrit dans le cadre du module *Langage Web 2 — XML* du Master 1 GIL de l'Université de Rouen Normandie. Il constitue l'aboutissement de quatre travaux pratiques portant successivement sur :

- La définition de schémas **XSD** pour la validation de flux XML SEPA
- La conception de feuilles de style **XSLT** pour la transformation XML vers HTML
- Le développement d'un service **REST** avec Spring Boot
- L'intégration complète et le déploiement sur machine virtuelle

1.2 Objectifs

L'application **SEPA26** est un service REST permettant de gérer des transactions bancaires au format *SEPA (Single Euro Payments Area)*, en s'appuyant sur une implémentation partielle de la norme **ISO 20022**. Les objectifs principaux sont :

- Exposer une API REST complète (GET, POST, DELETE) pour la gestion des transactions SEPA
- Valider les flux XML entrants contre un schéma XSD
- Transformer les flux XML en pages HTML via XSLT
- Assurer la persistance des données en base PostgreSQL
- Fournir un outil de transfert ergonomique (page web)
- Déployer l'application sur une machine virtuelle via Docker et Jenkins

1.3 Architecture générale

L'application repose sur une architecture en couches classique Spring Boot :

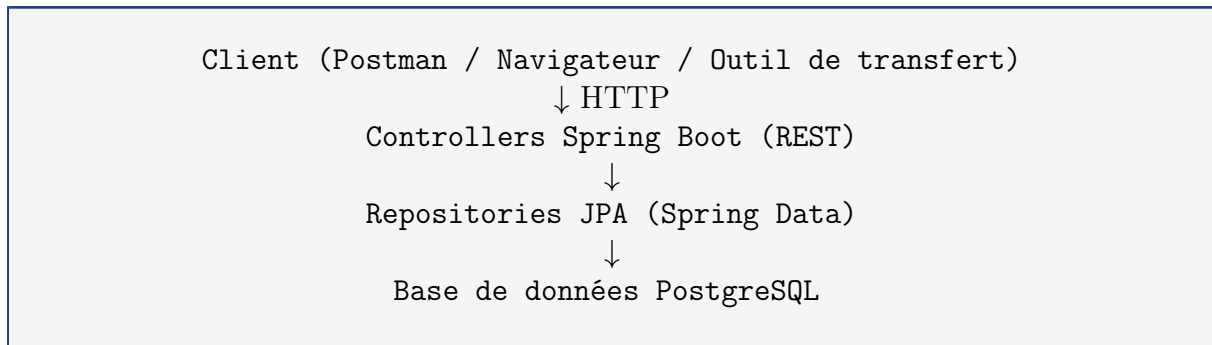


FIGURE 1.1 – Architecture en couches de l’application SEPA26

1.4 Technologies utilisées

Technologie		Rôle dans le projet
Java 21		Langage principal du service REST
Spring Boot 4.0.3		Framework de développement du service
Spring Data JPA		Couche de persistance et accès base de données
JAXB (Jakarta XML Bind)		Sérialisation/désérialisation XML ↔ Java
XSD (XML Schema)		Validation des flux SEPA reçus
XSLT 3.0 (Saxon-HE)		Transformation XML → HTML
PostgreSQL 15		Base de données relationnelle
Docker		Conteneurisation de l’application
Docker Compose		Orchestration des conteneurs (app + BDD)
Jenkins		Pipeline CI/CD
Maven		Gestion des dépendances et build
Postman		Tests et validation de l’API REST

2. Modélisation XML et validation XSD

2.1 Structure d'un flux SEPA26

Un flux SEPA26 est un document XML respectant une structure hiérarchique en trois niveaux, inspirée de la norme ISO 20022 :

- **Niveau 1** — **Document / GrpHdr** : L'en-tête global du fichier contenant l'identifiant du message, la date de création, le nombre de transactions et le montant total de contrôle.
- **Niveau 2** — **PmtInf** : Le lot de paiement, regroupant plusieurs transactions vers un même créancier à une même date de collecte.
- **Niveau 3** — **DrctDbtTxInf** : La transaction individuelle de prélèvement, contenant le montant, les informations du débiteur, son IBAN et son BIC.

2.2 Le schéma XSD

2.2.1 Rôle et conception

Le fichier `sepa26.xsd` définit les règles de validation applicables à tout flux SEPA reçu par le service. Il est placé dans `src/main/resources/` afin d'être accessible au moment de l'exécution.

2.2.2 Types simples définis

Listing 2.1 – Types simples du schéma XSD

```
<xs:simpleType name="Max35Text">
  <xs:restriction base="xs:string">
    <xs:minLength value="1"/>
    <xs:maxLength value="35"/>
    <xs:pattern value="[A-Za-z0-9_/?:( ) . , ' + - ] +"/>
  </xs:restriction>
</xs:simpleType>

<xs:simpleType name="IBANType">
  <xs:restriction base="xs:string">
    <xs:pattern value="[A-Z]{2}[0-9]{2}[A-Z0-9]{1,30}"/>
  </xs:restriction>
</xs:simpleType>
```

```
<xs:simpleType name="BICType">
  <xs:restriction base="xs:string">
    <xs:pattern value="[A-Z]{4}[A-Z]{2}[A-Z0-9]{2}([A-Z0-9]{3})
      ?"/>
  </xs:restriction>
</xs:simpleType>
```

Ces types garantissent que :

- Les identifiants textuels ne dépassent pas 35 caractères et n'utilisent que des caractères autorisés (norme ISO 20022)
- Les IBAN respectent le format international (2 lettres de pays + 2 chiffres de contrôle + jusqu'à 30 caractères alphanumériques)
- Les BIC respectent le format SWIFT standard (8 ou 11 caractères)

2.2.3 Namespace et espace de noms

Le schéma déclare le namespace `http://univ.fr/sepa26` comme espace de noms cible. Tous les éléments du document XML doivent appartenir à cet espace de noms pour être validés. Côté Java, ce namespace est déclaré dans `package-info.java` :

```
1 @jakarta.xml.bind.annotation.XmlSchema(
2     namespace = "http://univ.fr/sepa26",
3     elementFormDefault = jakarta.xml.bind.annotation.XmlNsForm.
4     QUALIFIED
5 )
6 package fr.univrouen.sepa26.model;
```

Listing 2.2 – Déclaration du namespace JAXB

2.2.4 Validation dans le service

La validation XSD est effectuée dans le `PostController` avant toute tentative d'insertion en base :

```
1 SchemaFactory factory =
2     SchemaFactory.newInstance(XMLConstants.W3C_XML_SCHEMA_NS_URI);
3 Source schemaFile = new StreamSource(
4     getClass().getResourceAsStream("/sepa26.xsd"));
5 Schema schema = factory.newSchema(schemaFile);
6 Validator validator = schema.newValidator();
7
8 // L ve une SAXException si le flux est invalide
9 validator.validate(new StreamSource(new StringReader(xmlPayload)));
```

Listing 2.3 – Validation XSD à la réception du flux

En cas d'échec de validation, une réponse XML d'erreur est retournée avec la description précise de la première erreur détectée, sans aucune modification de la base de données.

3. Modèle de données Java et persistance JPA

3.1 Mapping objet-relationnel

Le modèle Java reflète fidèlement la structure hiérarchique du flux SEPA26. Chaque niveau de la hiérarchie XML correspond à une entité JPA persistée en base de données.

3.1.1 Les entités principales

Classe Java	Table PostgreSQL	Rôle
GroupHeader	group_header	En-tête du fichier SEPA
PaymentInformation	payment_information	Lot de paiement
DirectDebitTransaction	direct_debit_transaction	Transaction individuelle

3.1.2 Relations entre entités

Les relations entre entités sont gérées par JPA avec sauvegarde en cascade :

```
1 @OneToMany(cascade = CascadeType.ALL, mappedBy = "groupHeader")
2 @XmlTransient
3 private List<PaymentInformation> paymentInformationList;
```

Listing 3.1 – Relation OneToMany avec cascade dans GroupHeader

```
1 @ManyToOne
2 @JoinColumn(name = "payment_information_id")
3 @XmlTransient
4 private PaymentInformation paymentInformation;
```

Listing 3.2 – Clé étrangère dans DirectDebitTransaction

L'annotation `@XmlTransient` est cruciale : elle indique à JAXB d'ignorer ces champs lors de la sérialisation XML, évitant ainsi les boucles infinies dues aux relations bidirectionnelles.

3.1.3 Double annotation JPA + JAXB

Une des complexités du projet est que les mêmes classes Java sont utilisées à la fois pour la persistance (JPA) et pour la sérialisation XML (JAXB). Chaque champ porte donc potentiellement deux types d'annotations :


```
1 @XmlElement(name = "PmtInfId")    // JAXB : balise XML
2 private String pmtInfId;
3
4 @XmlElement(name = "PmtTpInf")    // JAXB : balise XML
5 @Embedded                        // JPA : integration dans la table
6 private PaymentTypeInformation pmtTpInf;
```

Listing 3.3 – Exemple de double annotation JPA + JAXB

3.1.4 Ordre des éléments XML avec @XmlType

La norme SEPA impose un ordre strict des balises XML. L'annotation @XmlType(propOrder = {...}) garantit que JAXB respecte cet ordre lors de la sérialisation :

```
1 @XmlType(propOrder = {
2     "pmtId", "instdAmt", "drctDbtTx",
3     "dbtrAgt", "dbtr", "dbtrAcct", "rmtInf"
4 })
5 public class DirectDebitTransaction { ... }
```

Listing 3.4 – Contrôle de l'ordre des balises XML

4. Service REST — Endpoints et contrôleurs

4.1 Vue d'ensemble des endpoints

URL	Méthode	Description
/	GET	Page d'accueil HTML
/help	GET	Documentation des endpoints
/sepa26/resume/xml	GET	Liste des 10 dernières transactions (XML)
/sepa26/resume/html	GET	Liste des 10 dernières transactions (HTML)
/sepa26/xml/{id}	GET	Détail d'une transaction (XML conforme XSD)
/sepa26/html/{id}	GET	Détail d'une transaction (HTML via XSLT)
/sepa26/insert	POST	Insertion avec validation XSD
/sepa26/delete/{id}	DELETE	Suppression avec nettoyage en cascade
/sepa26/search	GET	Recherche par date et/ou montant (bonus)
/transfert	GET	Outil de transfert de fichiers

4.2 Contrôleurs

4.2.1 GetController — Consultation des données

Le `GetController` gère l'ensemble des routes de lecture. Il illustre plusieurs techniques importantes :

Résumé XML avec DTO

Plutôt que de retourner directement les entités JPA, un objet de transfert dédié (*Data Transfer Object*) est utilisé pour le résumé :

```
1 @XmlElement(name = "Resume")
2 @XmlAccessorType(XmlAccessType.FIELD)
3 public class ResumeTransactionDTO {
4     @XmlElement(name = "Transaction")
```

```

5     private List<TransactionSimple> transactions;
6
7     public static class TransactionSimple {
8         @XmlElement(name = "Id")         private Long id;
9         @XmlElement(name = "CreDtTm")    private String creDtTm;
10        @XmlElement(name = "PmtId")     private String pmtId;
11        @XmlElement(name = "CtrlSum")    private String ctrlSum;
12    }
13 }

```

Listing 4.1 – DTO pour le résumé XML

Cette approche présente deux avantages : elle découple la représentation API du modèle interne, et elle laisse JAXB gérer proprement la sérialisation XML sans risque de boucle infinie.

Détail XML conforme au XSD

Pour la route `/sepa26/xml/{id}`, le flux retourné doit être conforme au XSD. Cela nécessite de reconstruire un objet Document complet à partir de la transaction récupérée :

```

1 DirectDebitTransaction tx = txOpt.get();
2 Document wrapper = new Document();
3 CustomerDirectDebitInitiation sepa = new
    CustomerDirectDebitInitiation();
4
5 sepa.setGrpHdr(tx.getPaymentInformation().getGroupHeader());
6
7 PaymentInformation pmtInf = tx.getPaymentInformation();
8 pmtInf.setDrctDbtTxInf(Collections.singletonList(tx));
9
10 sepa.setPmtInf(Collections.singletonList(pmtInf));
11 wrapper.setCustomerDirectDebitInitiation(sepa);
12
13 return ResponseEntity.ok(wrapper); // Spring + JAXB serialize en
    XML

```

Listing 4.2 – Reconstruction du Document XML complet

Détail HTML via transformation XSLT

La route `/sepa26/html/{id}` applique une transformation XSLT au XML généré :

```

1 // 1. Marshalling de l'objet Java en XML
2 JAXBContext context = JAXBContext.newInstance(
    DirectDebitTransaction.class);
3 Marshaller marshaller = context.createMarshaller();
4 StringWriter xmlWriter = new StringWriter();
5 marshaller.marshal(txOpt.get(), xmlWriter);
6
7 // 2. Chargement du fichier XSLT
8 InputStream xsltStream = getClass().getResourceAsStream("/sepa26.
    xslt");

```

```

9
10 // 3. Transformation avec Saxon (XSLT 3.0)
11 TransformerFactory factory = new net.sf.saxon.
    TransformerFactoryImpl();
12 Transformer transformer = factory.newTransformer(
13     new StreamSource(xsltStream));
14 transformer.transform(
15     new StreamSource(new StringReader(xmlWriter.toString())),
16     new StreamResult(htmlWriter));

```

Listing 4.3 – Transformation XSLT en Java

L'utilisation de **Saxon-HE** est justifiée par la version XSLT 3.0 utilisée dans la feuille de style, qui utilise des fonctions comme `current-date()` non supportées par le processeur Xalan inclus par défaut dans Java.

4.2.2 PostController — Insertion avec validation

Le `PostController` implémente un pipeline de traitement en cinq étapes :

1. **Validation XSD** : Le flux reçu est validé contre le schéma avant tout traitement
2. **Désérialisation JAXB** : Le XML valide est transformé en objets Java
3. **Vérification des doublons** : Le `MsgId` est vérifié en base pour éviter les insertions dupliquées
4. **Liaison des relations** : Les relations bidirectionnelles sont établies manuellement
5. **Sauvegarde en cascade** : Un seul `save()` sur le `GroupHeader` déclenche la sauvegarde de tous les niveaux

```

1 if (groupHeaderRepository.existsByMsgId(msgId)) {
2     return ResponseEntity.ok(
3         "<Response><status>ERROR</status>" +
4         "<description>MsgId '" + msgId +
5         "' existe deja</description></Response>"
6     );
7 }

```

Listing 4.4 – Vérification des doublons par `MsgId`

4.2.3 DeleteController — Suppression avec nettoyage

La suppression d'une transaction respecte la contrainte du cahier des charges : tout lot ou en-tête devenu vide après suppression est également supprimé.

```

1 transactionRepository.delete(tx);
2
3 // Nettoyage du lot si vide
4 lot.getTransactions().remove(tx);
5 if (lot.getTransactions().isEmpty()) {
6     paymentInformationRepository.delete(lot);
7 }

```

```
8      // Nettoyage de l'en-tête si vide
9      header.getPaymentInformationList().remove(lot);
10     if (header.getPaymentInformationList().isEmpty()) {
11         groupHeaderRepository.delete(header);
12     }
13 }
```

Listing 4.5 – Nettoyage en cascade lors de la suppression

L'annotation `@Transactional` est indispensable sur cette méthode pour que toutes ces opérations s'exécutent dans une seule transaction de base de données, garantissant la cohérence en cas d'erreur.

5. Transformation XSLT

5.1 Rôle de la feuille de style

La feuille de style `sepa26.xslt` (version 3.0) prend en entrée un flux XML SEPA26 et produit une page HTML complète présentant le détail de la transaction de manière lisible.

5.2 Fonctionnalités clés

5.2.1 Gestion du namespace

Le namespace `http://univ.fr/sepa26` est déclaré avec le préfixe `p`, ce qui permet de cibler précisément les éléments du flux :

Listing 5.1 – Déclaration du namespace dans le XSLT

```
<xsl:stylesheet version="3.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  xmlns:p="http://univ.fr/sepa26"
  exclude-result-prefixes="p xs">
```

5.2.2 Date dynamique avec XSLT 3.0

La date d'émission du document est calculée dynamiquement grâce à la fonction `current-date()` de XSLT 3.0, avec un formatage personnalisé du mois en français :

Listing 5.2 – Calcul et formatage de la date en XSLT

```
<xsl:variable name="aujourd'hui" select="current-date()"/>
<xsl:variable name="moisNum" select="month-from-date($aujourd'hui)"/>
<xsl:variable name="moisTexte">
  <xsl:choose>
    <xsl:when test="$moisNum=1">Janv</xsl:when>
    <xsl:when test="$moisNum=5">Mai</xsl:when>
    <!-- ... -->
  </xsl:choose>
</xsl:variable>
```

5.2.3 Gestion du choix BIC / Agent

La feuille de style gère élégamment le cas où une banque débitrice est identifiée soit par un BIC, soit par un identifiant alternatif :

Listing 5.3 – Gestion du choix BIC ou Othr en XSLT

```
<xsl:choose>
  <xsl:when test="p:DbtrAgt/p:FinInstnId/p:BIC">
    <td class="label">BIC</td>
    <td class="valeur">
      <xsl:value-of select="p:DbtrAgt/p:FinInstnId/p:BIC"/>
    </td>
  </xsl:when>
  <xsl:otherwise>
    <td class="label">Agent</td>
    <td class="valeur">
      <xsl:value-of select="p:DbtrAgt/p:FinInstnId/p:Othr/
        p:Id"/>
    </td>
  </xsl:otherwise>
</xsl:choose>
```

5.2.4 Tri et sommaire

La feuille de style génère automatiquement un sommaire triant les transactions par montant décroissant :

Listing 5.4 – Tri des transactions par montant

```
<xsl:for-each select="//p:DrctDbtTxInf">
  <xsl:sort select="p:InstdAmt" data-type="number" order="
    descending"/>
  <li>
    montant = <xsl:value-of select="p:InstdAmt"/>
    <xsl:value-of select="p:InstdAmt/@Ccy"/>
  </li>
</xsl:for-each>
```

6. Déploiement et intégration continue

6.1 Conteneurisation Docker

L'application est conteneurisée via Docker pour garantir la reproductibilité du déploiement. Le `Dockerfile` utilise une image Alpine légère avec Java 21 JRE :

Listing 6.1 – Dockerfile de l'application

```
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S spring && adduser -S spring -G spring
ARG JAR_FILE=target/sepa26server.jar
COPY ${JAR_FILE} /opt/sepa26server.jar
USER spring:spring
WORKDIR /opt
ENTRYPOINT ["java", "-jar", "sepa26server.jar"]
EXPOSE 8100
```

L'utilisation d'un utilisateur non-root (`spring`) respecte les bonnes pratiques de sécurité pour les conteneurs en production.

6.2 Docker Compose

Le fichier `compose.yaml` orchestre deux services : l'application Spring Boot et la base de données PostgreSQL :

Listing 6.2 – Orchestration avec Docker Compose

```
services:
  database:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: sepa26
    volumes:
      - sepa26_db_data:/var/lib/postgresql/data

  sepa26_app:
    image: sepa26-server
    ports:
      - "8100:8100"
    environment:
      - DB_HOST=database
    depends_on:
      - database
```


La variable d'environnement `DB_HOST` permet de configurer dynamiquement l'URL de connexion à la base, avec un fallback sur `localhost` pour le développement local :

Listing 6.3 – Configuration dynamique de la BDD

```
spring.datasource.url=jdbc:postgresql://${DB_HOST:localhost}:5432/
sepa26
```

6.3 Pipeline Jenkins

Un pipeline Jenkins a été mis en place pour automatiser le cycle de build et déploiement. Déclenché par un push sur la branche `master` du dépôt Git, il exécute :

1. La compilation Maven : `./mvnw clean package`
2. L'exécution des tests unitaires
3. La construction de l'image Docker
4. Le déploiement via Docker Compose sur la VM
5. La vérification de l'accessibilité de la page d'accueil

7. Fonctionnalités bonus

7.1 API de recherche

Une route de recherche a été implémentée en bonus, permettant de filtrer les transactions selon deux critères combinables :

- **date** : Retourne les transactions dont la date **CreDtTm** est identique ou postérieure à la date indiquée
- **sum** : Retourne les transactions dont le montant **InstdAmt** est égal ou supérieur au montant indiqué

Exemples d'utilisation :

```
GET /sepa26/search?sum=100
GET /sepa26/search?date=2023-01-01
GET /sepa26/search?date=2023-01-01&sum=50
```

La requête JPQL dans le repository gère l'optionnalité des paramètres grâce au test `:param IS NULL` :

```
1 @Query("SELECT t FROM DirectDebitTransaction t WHERE " +
2     "(:date IS NULL OR " +
3     "t.paymentInformation.groupHeader.creDtTm >= :date) AND " +
4     "(:sum IS NULL OR t.instdAmt.value >= :sum)")
5 List<DirectDebitTransaction> searchTransactions(
6     @Param("date") String date,
7     @Param("sum") Double sum);
```

Listing 7.1 – Requête JPQL de recherche avec filtres optionnels

Si la recherche ne retourne aucun résultat, la réponse XML contient `<status>NONE</status>`.

7.2 Gestion enrichie des erreurs

Conformément au bonus II.1 du cahier des charges, les messages d'erreur XML incluent un champ `<description>` détaillant la cause de l'erreur :

Listing 7.2 – Message d'erreur enrichi

```
<Response>
  <status>ERROR</status>
  <description>Flux XML non valide : cvc-pattern-valid:
    'INVALID_IBAN' n'est pas valide pour le type 'IBANType'
  </description>
</Response>
```

7.3 Outil de transfert de fichiers

Un outil de transfert ergonomique a été développé sous forme de page web accessible à l'adresse `/transfert`. Il permet :

- De configurer dynamiquement l'URL du service REST (URL + port paramétrables)
- De sélectionner un fichier XML sur le système local
- D'envoyer le fichier vers le service via une requête POST asynchrone (Fetch API)
- D'afficher la réponse du serveur (succès ou erreur) directement dans l'interface

Cette page utilise JavaScript côté client pour effectuer la requête HTTP sans rechargement de page, offrant une expérience utilisateur fluide.

7.4 Journalisation (Logging)

Un système de logs a été intégré dans les contrôleurs principaux grâce à SLF4J :

```
1 private static final Logger log =  
2     LoggerFactory.getLogger(PostController.class);  
3  
4 log.info("Reception d'une nouvelle requete d'insertion SEPA...");  
5 log.error("Echec de l'insertion : " + e.getMessage());
```

Listing 7.3 – Exemple de logging dans PostController

8. Tests et validation Postman

8.1 Tests unitaires

Un test unitaire JUnit vérifie le bon fonctionnement du parsing JAXB pour un flux XML minimal mais valide :

```
1 @Test
2 public void testXmlParsing() {
3     String fauxXml = "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n"
4         +
5         "<Document xmlns=\"http://univ.fr/sepa26\">\n" +
6         "    <CstmrDrctDbtInitn>\n" +
7         "        <GrpHdr>\n" +
8         "            <MsgId>TEST-UNIT-SEPA</MsgId>\n" +
9         "        </GrpHdr>\n" +
10        "    </CstmrDrctDbtInitn>\n" +
11        "</Document>";
12
13    JAXBContext context = JAXBContext.newInstance(Document.class);
14    Document doc = (Document) context.createUnmarshaller()
15        .unmarshal(new StringReader(fauxXml));
16
17    assertNotNull(doc);
18    assertEquals("TEST-UNIT-SEPA",
19        doc.getCustomerDirectDebitInitiation()
20            .getGrpHdr().getMsgId());
21 }
```

Listing 8.1 – Test unitaire de parsing XML

8.2 Validation Postman

Une collection Postman nommée **sepa26_amouchas** a été créée, regroupant toutes les requêtes nécessaires au test complet du service. Elle utilise une variable `{{url_server}}` pour permettre de basculer facilement entre l'environnement local et la VM de production.

Requête	Méthode	Description
Accueil HTML	GET	Vérification de la page d'accueil
Aide HTML	GET	Documentation des endpoints
Liste Résumé XML	GET	10 dernières transactions en XML

Requête		Méthode	Description
Liste HTML	Résumé	GET	10 dernières transactions en HTML
Détail XML	Transaction	GET	Flux XML conforme XSD pour id=1
Détail HTML	Transaction	GET	Page HTML via XSLT pour id=1
Insérer SEPA	un flux	POST	Insertion avec body XML complet
Supprimer transaction	une	DELETE	Suppression par identifiant
Recherche (Bonus)		GET	Filtre par montant (?sum=50)

9. Conclusion

9.1 Bilan des réalisations

Ce projet a permis de mettre en œuvre l'ensemble des technologies abordées lors des travaux pratiques du module XML :

Points obligatoires réalisés :

- Page d'accueil et page d'aide HTML
- Liste résumée des transactions en XML et HTML
- Détail d'une transaction en XML (conforme XSD) et HTML (via XSLT)
- Insertion de transactions avec validation XSD
- Suppression de transactions avec nettoyage en cascade
- Gestion des erreurs (XML et HTML selon le contexte)
- Déploiement sur VM avec Docker et Jenkins

Points bonus réalisés :

- API de recherche `/sepa26/search?date=&sum=` (II.2)
- Messages d'erreur XML enrichis avec description (II.1)
- Outil de transfert ergonomique accessible depuis le navigateur (III.1)
- Journalisation avec SLF4J
- Collection Postman complète avec variable serveur

9.2 Difficultés rencontrées

Les principales difficultés techniques ont été :

1. **La sérialisation JAXB avec JPA** : Les relations bidirectionnelles entre entités JPA créent des risques de boucles infinies lors de la sérialisation XML. La solution a été l'utilisation systématique de `@XmlTransient` sur les champs de relation inverse.
2. **La conformité XSD du XML retourné** : Retourner un flux XML conforme au XSD pour `/sepa26/xml/{id}` a nécessité de reconstruire manuellement un objet `Document` complet, car une `DirectDebitTransaction` isolée ne peut pas constituer un document SEPA valide.

3. **L'intégration de Saxon** : L'utilisation de XSLT 3.0 a nécessité d'ajouter la dépendance Saxon-HE et de forcer son utilisation via `new net.sf.saxon.TransformerFactoryImpl()`, Java intégrant par défaut un processeur XSLT 1.0/2.0 (Xalan).

9.3 Perspectives d'amélioration

Parmi les pistes d'amélioration possibles :

- Migration vers Thymeleaf pour les vues HTML (éviter les `StringBuilder`)
- Pagination sur la liste des transactions (au-delà des 10 dernières)

*Projet réalisé dans le cadre du M1 GIL
Université de Rouen Normandie — 2025/2026*
